Казанский федеральный университет
Институт информационных технологий
и интеллектуальных систем

Программная инженерия

Отчёт по блоку заданий 1

Развёртывание моделей машинного обучения
API и автоматическое тестирование
Рекомендательные системы

Выполнил
Баширов Адель Тагирович
Группа 11-411

Казань 2026

1 Цель и состав работы

В этой работе я использовал готовые модели для четырёх задач: анализа отзывов, распознавания звуков, классификации скриншотов и поиска объектов на видео. Затем добавил API для классификатора изображений и сравнил несколько способов рекомендовать фильмы. Код, инструкции запуска и результаты находятся в одном репозитории.

Классификатор сайтов правильно определил 38 из 45 страниц. API принимает изображение и возвращает результат в JSON. В задаче рекомендаций лучший NDCG@10 получен у item-CF: 0,1651. Кроме основных заданий, в проекте есть пример запуска небольшой языковой модели Qwen на ноутбуке.

Компонент	Решение	Папка
Текст	RuBERT — тональность отзывов	text/
Аудио	YAMNet — классы звуков ESC-10	audio/
Изображения	CLIP — тип веб-страницы	image/
Видео	YOLO11 — детекция объектов	video/
LLM	Qwen2.5-0.5B-Instruct	llm/
API	FastAPI, PyTest, GitHub Actions	api/, tests/
Рекомендации	Пять сравниваемых алгоритмов	recsys/

Данные и окружение

Текст, аудио и видео были готовы раньше. В отчёт включены их сохранённые результаты; при итоговой проверке для каждой модели выполнено отдельное предсказание. Полные оценки повторно не считались. Скриншоты, эксперименты Images и RecSys, проверка API и запуск Qwen относятся к 12 октября 2026 года.

В работе используются два ML-фреймворка: PyTorch для текста и изображений, TensorFlow для звука. Transformers упрощает загрузку моделей и подготовку входа, TensorFlow Hub даёт готовую YAMNet, а Ultralytics — средства для детекции и разметки кадров. Основное окружение: Python 3.11, macOS, Apple MPS. Аудио и видео запускаются в отдельных окружениях; версии сохранены в verification/environment-macos.txt.

Репозиторий

https://github.com/AdelBashiroff/ProgIngenering-s1sz1

В README каждого модуля описаны ТЗ, установка и запуск. Таблицы предсказаний и метрики сохранены в папках results. Большие веса моделей и исходные наборы данных скачиваются отдельно.

2 Архитектура проекта

Проект состоит из отдельных Python-модулей. Каждый получает свой тип данных и вызывает готовую модель. Результаты оценки сохраняются в CSV и JSON, графики — в PNG. Базы данных и общего веб-интерфейса здесь нет. Через HTTP доступен классификатор скриншотов, остальные модули запускаются из командной строки.

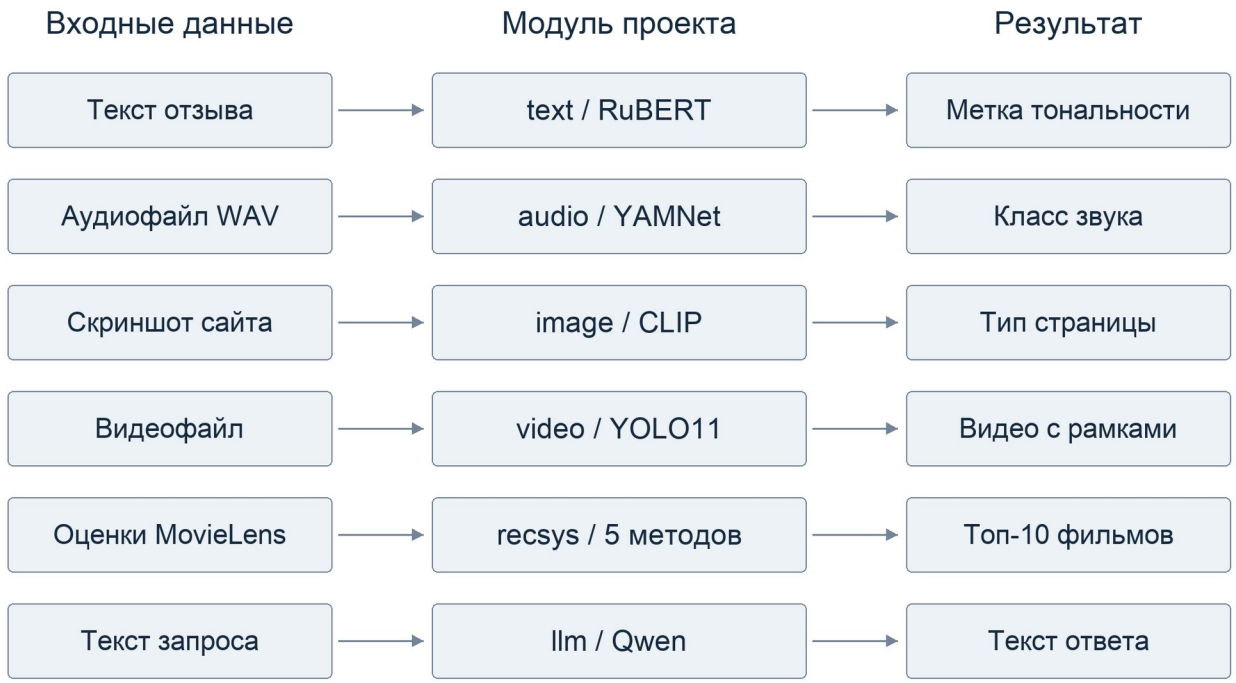


Рисунок 1 — Модули проекта и их входы и выходы

API использует ту же функцию predict, что и локальный классификатор. После проверки файла изображение переводится в RGB и передаётся CLIP. Веса и текстовые векторы классов кешируются. Поэтому модель не нужно заново загружать для каждого запроса.

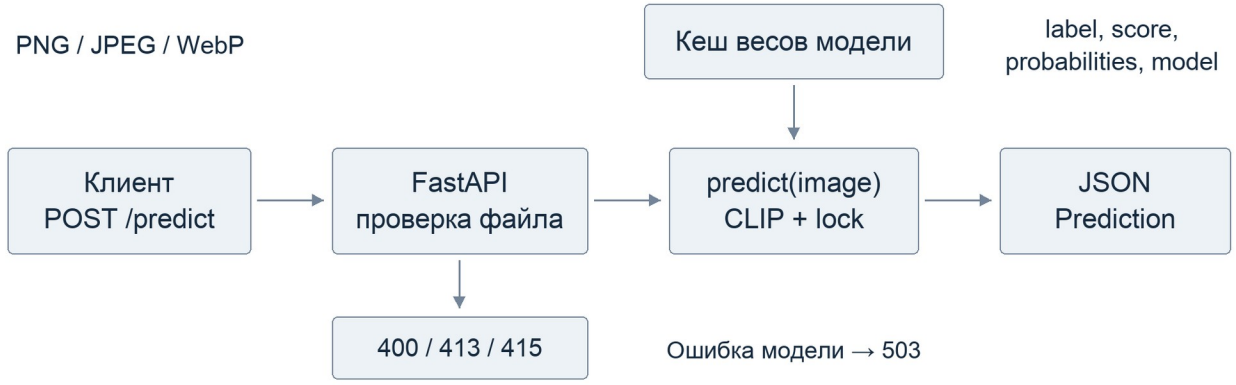


Рисунок 2 — Обработка запроса к классификатору

Схема показывает основной успешный путь и ошибки внутри обработчика. Если поле file не передано, FastAPI возвращает 422 до вызова обработчика. GET /health проверяет только процесс сервера; для проверки весов нужен запрос к /predict.

3 Стек и организация запуска

Для всех компонентов используется Python 3.11. Основные эксперименты выполнены на macOS; PyTorch использует Apple MPS, если он доступен. Таблица показывает версии проверенного окружения, а не минимально допустимые версии из requirements.txt.

Область	Технологии и версии	Назначение
Текст, Images, LLM	PyTorch 2.14.1; Transformers 4.57.6	Готовые веса, токенизация, обработка изображений, расчёт предсказаний
Аудио	TensorFlow 2.21.0; TensorFlow Hub 0.16.1	Загрузка YAMNet и классификация звуков
Видео	Ultralytics 8.4.171; OpenCV 5.0.0.93	YOLO11, чтение и сохранение кадров
HTTP API	FastAPI 0.143.0; Uvicorn 0.54.0; Pydantic 2.14.1	Маршруты сервера, обработка запросов, схема ответа
Работа с данными	NumPy 2.4.6; pandas 2.3.3; SciPy 1.17.1	Таблицы, векторы и разреженные матрицы
Оценка и RecSys	scikit-learn 1.9.1; Matplotlib 3.11.2	Метрики, TF-IDF, косинусное сходство и графики
Изображения и сбор	Pillow 12.3.0; Playwright 1.63.0	Проверка файлов и получение скриншотов в Chromium
Тесты	PyTest 9.1.1; HTTPX 0.28.1	Проверка контракта API и расчётов без скачивания весов
Контроль версий и CI	Git; GitHub Actions; Python 3.11 в CI	Хранение кода и автоматический запуск тестов

У аудио и видео отдельные виртуальные окружения: зависимости TensorFlow и Ultralytics устанавливаются из README соответствующего модуля. Для Images, API, RecSys и LLM достаточно корневого requirements-dev.txt. В CI устанавливаются только пакеты, нужные тестам; нейросети там не запускаются.

Где находится реализация

Репозиторий проекта

Зависимости основного окружения

Автотесты

Сценарий GitHub Actions

Результаты реальных запусков и проверок

4 Анализ тональности текста

Для текста я выбрал анализ русскоязычных отзывов. На вход подаётся строка, на выходе нужны positive, negative или neutral и оценка score. Используется seara/rubert-tiny2-russian-sentiment. Токенизатор разбивает текст на части, BERT учитывает их контекст, а классификатор выбирает одну из трёх меток. Вход ограничен 256 токенами, дообучения нет [3].

В собственном CSV-наборе 151 отзыв, две колонки: text и label. Я сравнил RuBERT-tiny2 и RuBERT-base через один интерфейс Transformers. Accuracy показывает долю верных ответов, macro-F1 одинаково учитывает все три класса. Это удобно, когда число отзывов разных классов различается.

Модель	Accuracy	Macro-F1
RuBERT-tiny2	0,7219	0,7291
RuBERT-base	0,6689	0,6696

У tiny2 macro-F1 равен 0,7291, что выше выбранного порога 0,70. Но 95% доверительный интервал [0,6566; 0,8023] пересекает этот порог. Интервал разности с base включает ноль, поэтому уверенно объявить tiny2 точнее нельзя. Для проекта оставлена компактная tiny2: в сохранённом запуске она обрабатывала 222 текста/с против 169 у base.

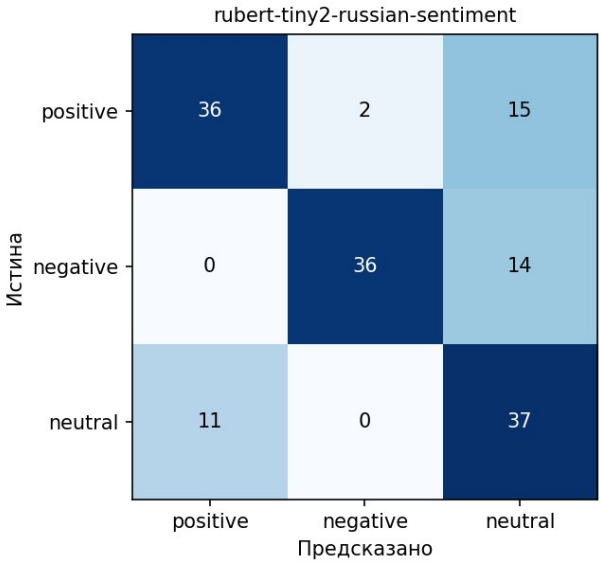


Рисунок 3 — Матрица ошибок RuBERT-tiny2 на 151 отзыве

Чаще возникают ошибки на нейтральных и смешанных отзывах: например, когда в одной фразе есть и похвала, и жалоба. Набор небольшой и собран вручную, поэтому эти цифры нельзя переносить на любые отзывы.

При повторной проверке фраза «Отличный сервис, всё понравилось!» получила positive со score 0,9964. Этот пример подтверждает, что модель запускается; её качество оценивалось на всём наборе.

```
python text/sentiment.py "Отличный сервис, всё понравилось!"
```

5 Классификация звуков

Для звука выбрана YAMNet из TensorFlow Hub: у неё уже есть веса и 521 класс AudioSet. Модель преобразует звук в спектрограмму и обрабатывает её свёрточной сетью MobileNet [4]. Hub удобен тем, что модель загружается вместе с таблицей классов. В этой работе WAV приводится к моно, float32 и 16 кГц, а оценки коротких фрагментов объединяются средним или максимумом.

Для проверки взят ESC-10: 400 WAV-записей по 5 секунд, по 40 примеров на каждый из десяти классов. В таблице метаданных хранятся имена файлов и метки. Названия ESC-10 сопоставлены с AudioSet вручную. В закрытой задаче ответ выбирается только из этих десяти классов. Ни mean, ни max не требуют обучения модели.

Агрегация	Accuracy	Macro-F1
mean	0,9425	0,9426
max	0,9400	0,9401

С усреднением mean получен macro-F1 0,9426, 95% доверительный интервал [0,9187; 0,9636]. Для accuracy интервал равен [0,9200; 0,9650]. Если разрешить модели выбирать из всех 521 классов, доля совпадений с сопоставленными метками падает до 0,2375. Получается, высокая точность здесь сильно зависит от ограничения списка ответов.

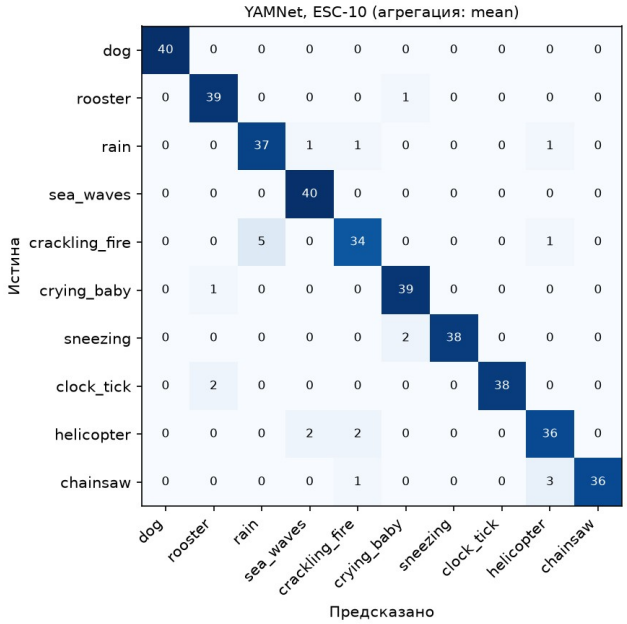


Рисунок 4 — Матрица ошибок YAMNet с агрегацией mean

По матрице ошибок видно, что модель иногда путает огонь, дождь и волны. Возможно, причина в похожем шуме. При повторной проверке файл 1-47273-A-21.wav получил метку sneezing со score 0,2812. Аудио загрузилось и было обработано без ошибки.

6 Детекция объектов на видео

Для видео я использовал YOLO11n и YOLO11s. Детектор обрабатывает каждый кадр и за один проход предсказывает рамки и классы объектов [5]. Ultralytics удобен готовыми весами и средствами запуска и оценки модели. Программа сохраняет видео с рамками, CSV с детекциями и JSON со статистикой. Слежение за отдельным объектом между кадрами не реализовано.

Качество проверено на 500 случайно выбранных изображениях COCO val2017, seed=42. COCO содержит изображения и JSON-разметку с классами и рамками объектов. Метрика AP@[0,5:0,95] учитывает разные пороги совпадения рамок IoU. Разметки для самого видеоролика нет, поэтому AP рассчитан только на изображениях.

Модель	**AP**	**AP50**	**AP small**	**мс/изобр.**
YOLO11n	0,415	0,571	0,226	35,1
YOLO11s	0,484	0,653	0,310	47,2

У YOLO11s AP выше на 0,069, особенно заметна разница для мелких объектов. При этом она работает медленнее. Для видео оставлена YOLO11n. В сохранённом эксперименте выполнены оба критерия ТЗ: AP не ниже 0,30 и скорость не ниже 5 кадров/с.

Характеристика видео	**Результат**
Разрешение и частота	3840×2160, 59,91 кадра/с
Длительность и объём	29,48 с; 1766 кадров
Скорость полной обработки	13,0 кадров/с
Инференс модели	21,5 мс/кадр
Всего покадровых детекций	48 207
Люди / автомобили	24 204 / 23 576 детекций

Полная обработка идёт со скоростью 13 кадров/с при частоте исходного ролика около 60 кадров/с, то есть не в реальном времени. Время тратится также на чтение, отрисовку и запись 4K-кадров. 48 207 детекций не означают столько же разных объектов: одна машина может попасть в сотни кадров.

Сложнее всего обнаруживать мелкие и частично закрытые объекты в плотных группах. Можно попробовать большее входное разрешение или более крупную модель, но тогда обработка станет медленнее. Отдельных экспериментов с этими изменениями не было.

Для повторной проверки использовано изображение COCO 000000047571.jpg. Найдены три объекта, в том числе horse с confidence 0,9127 и person с confidence 0,8315. Ссылка на исходный ролик находится в video/videos/my_video.txt. Сам ролик заново целиком не обрабатывался.

7 Классификация скриншотов сайтов

Для Images я выбрал определение типа сайта по скриншоту: landing, shop или blog. CLIP ViT-B/32 сравнивает векторы изображения и текстовых описаний классов [2]. Это позволяет задать свои классы без дообучения. Transformers загружает модель и её обработчик входа, а PyTorch выполняет расчёт. В API используется заранее выбранный вариант pad + descriptive.

Из 73 адресов удалось получить 56 скриншотов, ещё 17 страниц не загрузились. При просмотре исключены 11 кадров с заглушками и окнами, закрывающими содержимое. Осталось 45 изображений: 18 лендингов, 9 магазинов и 18 блогов. Отбор сделан до оценки модели, по качеству снимка.

Скриншоты сняты в Chromium при размере окна 1440×900 и сохранены в JPEG. В manifest.csv записаны адрес, метка, путь к файлу, время, HTTP-статус, SHA256 и результат проверки. Сравниваются обрезка crop и дополнение полями pad, а также три набора описаний классов. Все принятые снимки есть в GitHub.

Обработка	Промпты	Accuracy	Macro-F1
pad	ensemble	0,8667	0,8667
crop	descriptive	0,8444	0,8500
pad	descriptive	0,8444	0,8436
crop	ensemble	0,8444	0,8430
crop	simple	0,7778	0,7778
pad	simple	0,7556	0,7525

Основной вариант распознал 38 из 45 страниц: accuracy 0,8444, macro-F1 0,8436. Порог ТЗ 0,70 выполнен; 95% доверительный интервал F1 [0,7159; 0,9363] рассчитан по 1000 bootstrap-повторам. Лучший результат у pad + ensemble — 0,8667. Для подтверждения преимущества нужен новый набор: сейчас варианты сравнивались на тех же данных, где оценивались.

Класс	Precision	Recall	F1	N
landing	0,7727	0,9444	0,8500	18
shop	0,8000	0,8889	0,8421	9
blog	1,0000	0,7222	0,8387	18

8 Ошибки и ограничения Images

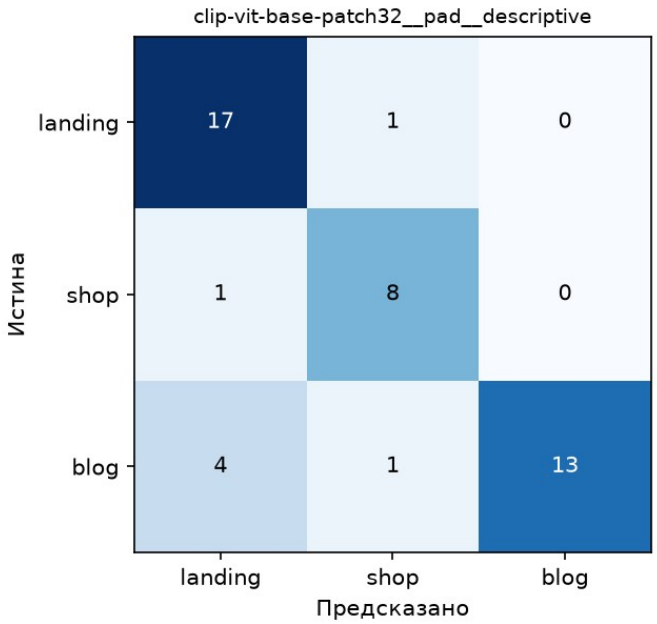


Рисунок 5 — Матрица ошибок основной конфигурации CLIP

Пять блогов модель отнесла к лендингам или магазину. На Stack Overflow и CSS-Tricks вверху есть крупные блоки, похожие на рекламу, а у Ed Zitron — платные подписки. SparkFun получил landing, Tilda — shop. По виду страниц эти ошибки можно объяснить, но точно установить, какие элементы повлияли на модель, по одному ответу нельзя.

На Stack Overflow модель ошиблась со score 0,9196. Поэтому даже высокий score нельзя считать гарантией правильного ответа.

Порог score	Охват	Accuracy
0,0	100,00%	84,44%
0,5	93,33%	88,10%
0,7	75,56%	97,06%
0,9	44,44%	95,00%

Если не принимать ответы со score ниже 0,7, точность на оставшихся страницах повышается до 97,06%. Однако примерно четверть страниц тогда остаётся без ответа. В API этот порог не включён: его ещё нужно проверить на других данных.

После проверки в наборе осталось 44 англоязычных сайта и только один русскоязычный. Сравнивать качество по языкам здесь нет смысла. Две страницы принадлежат Notion, а часть магазинов не удалось снять из-за блокировок. Ещё одно ограничение CLIP — уменьшение входа до 224×224: мелкий текст на странице теряется.

9 Примеры скриншотов и ответов модели

Оба изображения взяты из сохранённого набора, снятого 12 октября. Ниже показаны исходная метка и ответ основной конфигурации pad + descriptive. Примеры подобраны так, чтобы показать и правильный ответ, и ошибку с высоким score.

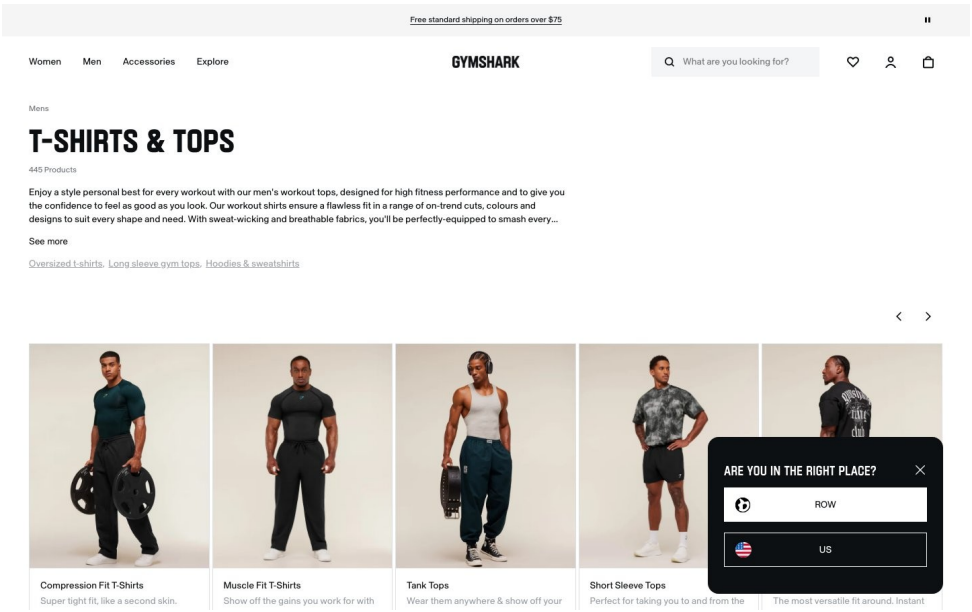


Рисунок 6 — Gymshark — метка shop, ответ shop, score 0,9856

В кадре видны карточки товаров и элементы каталога. Небольшое окно выбора региона закрывает часть правой карточки, но тип страницы остаётся понятен. Этот же файл использован для API. Адрес: gymshark.com/collections/t-shirts-tops/mens.

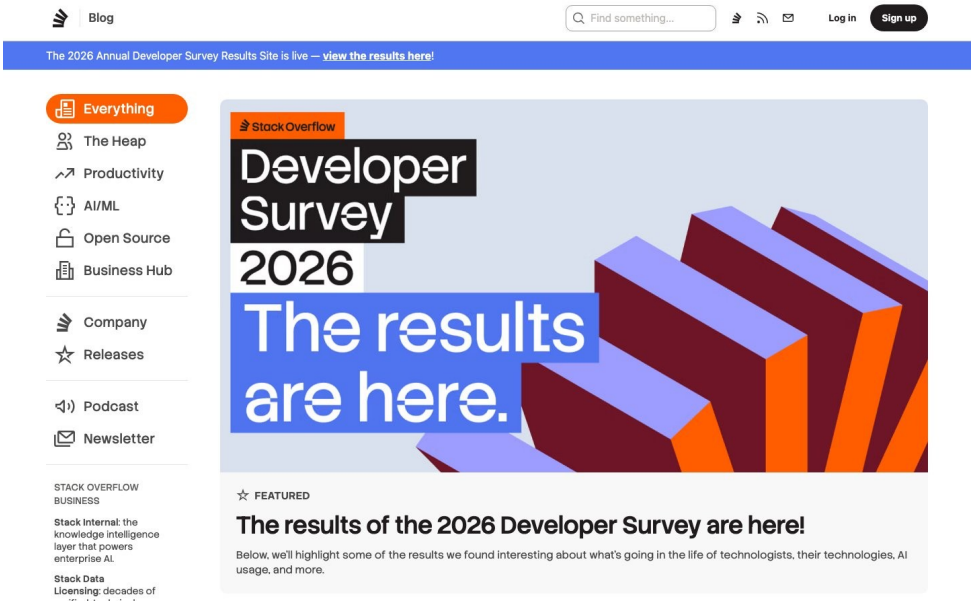


Рисунок 7 — Stack Overflow Blog — метка blog, ответ landing, score 0,9196

Верх страницы больше похож на промостраницу, чем на ленту статей. Даже при score 0,9196 ответ оказался неверным. Полная таблица, включая все ошибки, находится в image/results; исходные снимки и метки — в image/data.

10 API и автоматическое тестирование

Для доступа к CLIP я добавил FastAPI. POST /predict принимает файл и возвращает label, score, probabilities и имя модели. GET /health проверяет процесс, GET /model показывает настройки, а /docs открывает Swagger UI. Модель загружается при первом запросе. Блокировка не даёт двум запросам одновременно запустить расчёт.

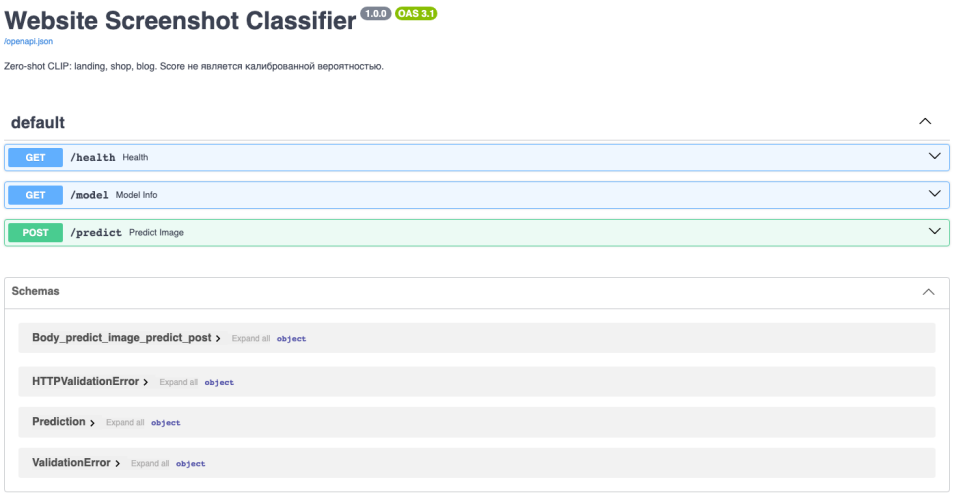


Рисунок 8 — Проверка API через Swagger UI

Код ответа	Условие
200	Корректное изображение и успешный инференс
400 / 415	Пустой или повреждённый файл / неподдерживаемый формат
413	Файл более 8 МиБ или изображение более 16 млн пикселей
422 / 503	Нет обязательного файла / модель недоступна

Реальный HTTP-запрос со скриншотом магазина завершился кодом 200: label=shop, score=0,9856; время 4.713 с с учётом первого обращения. Ответ сохранён в verification/api_real_request.json.

В проекте 29 тестов: они проверяют ответы API, сбор ссылок, разделение данных и расчёты RecSys, исключение просмотренных фильмов и холодный старт. В тестах API используется подмена модели через dependency_overrides, поэтому GitHub Actions не скачивает веса. Отдельно выполнен HTTP-запрос с настоящей CLIP.

GitHub Actions успешно выполнил тесты коммита a354731 (run 38358132244). Проверки запускаются после push, при pull request и вручную. API рассчитан на локальный запуск. При публикации сервера понадобится также ограничить полный размер HTTP-запроса на reverse proxy.

11 Фрагменты реализации Images и API

Ниже приведены строки из исходников проекта. Ссылки в заголовках ведут на соответствующие строки GitHub. При сборке отчёта код читается из файлов; убирается только общий отступ фрагмента.

Выбор класса по сходству векторов

Листинг 1 — image/clip_classifier.py, строки 136–143

```
probs = classify_features(encode_images([image], model_name, fit),
                          encode_prompts(prompts, model_name), logit_scale(model_name))[0]
best = int(probs.argmax())
return {
    "label": LABELS[best],
    "score": round(float(probs[best]), 4),
    "probabilities": {label: round(float(p), 4) for label, p in zip(LABELS, probs)},
}
```

Сначала получаются векторы изображения и описаний классов. classify_features считает softmax по их сходству, затем argmax выбирает метку. В ответ сохраняются score выбранного класса и значения всех трёх классов. Сам по себе softmax не гарантирует правильность ответа.

Проверка размера загрузки

Листинг 2 — api/main.py, строки 47–54

```
try:
    data = file.file.read(MAX_UPLOAD_BYTES + 1)
finally:
    file.file.close()
if len(data) > MAX_UPLOAD_BYTES:
    raise HTTPException(413, 'Изображение превышает лимит 8 МиБ')
if not data:
    raise HTTPException(400, 'Пустой файл')
```

Обработчик читает не больше лимита плюс один байт: этого достаточно, чтобы обнаружить превышение. Пустой файл даёт 400, слишком большой — 413. Дальше Pillow проверяет формат, декодирование и число пикселей. Ограничение всего HTTP-запроса до разбора multipart остаётся задачей серверной конфигурации.

Вызов модели и обработка сбоя

Листинг 3 — api/main.py, строки 69–75

```
try:
    # Сериализация инференса предотвращает несколько одновременных загрузок весов.
    with _inference_lock:
        result = predictor(image)
except (OSError, RuntimeError):
    raise HTTPException(503, 'Модель недоступна. Проверьте веса и ресурсы сервера.')
return Prediction(**result)
```

Блокировка не позволяет нескольким запросам одновременно загружать веса и запускать модель. При ошибке загрузки или выполнения возвращается 503, а успешный результат проходит проверку схемы Prediction.

12 Проверка API на реальном запросе

13 октября через Swagger UI отправлен JPEG из примера Gymshark. Сервер вернул 200, shop и score 0,9856. На снимке видны команда запроса, адрес, тело ответа и заголовки. Это ответ работающего сервера, а не пример, сгенерированный схемой OpenAPI.

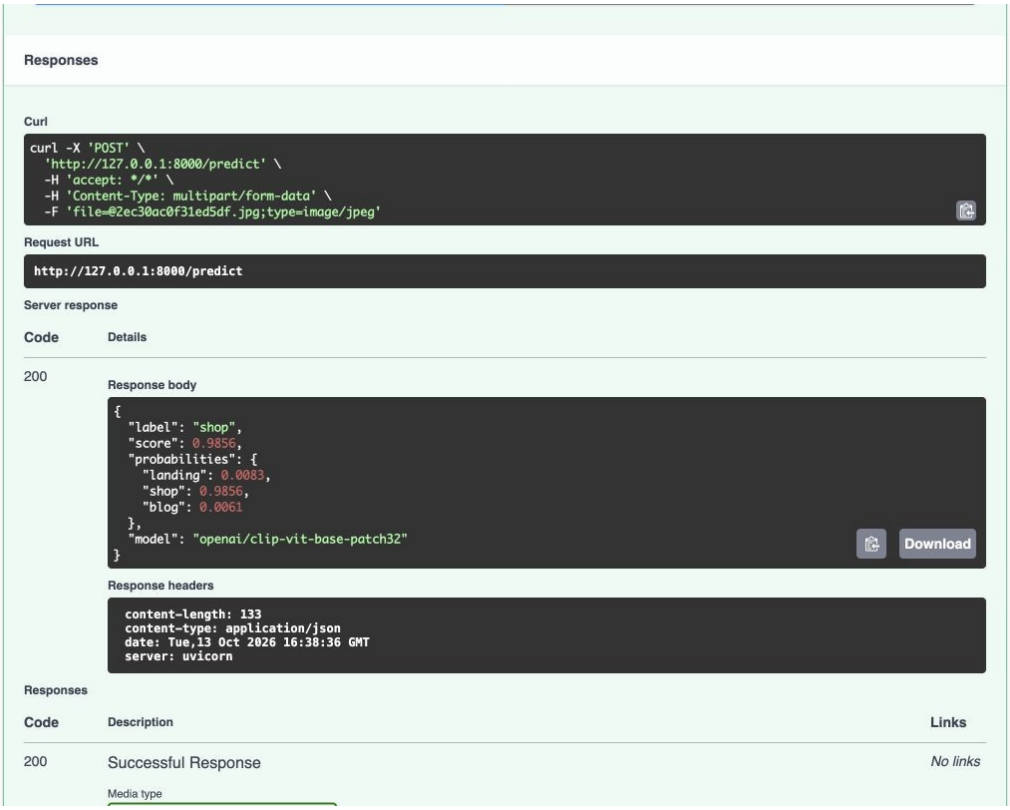


Рисунок 9 — Ответ CLIP на запрос из Swagger UI

Автоматическая проверка контракта

Листинг 4 — tests/test_api.py, строки 34–41

```
def test_predict_contract(client, fmt, mode):
    response = client.post('/predict', files={'file': ('screen', image_bytes(fmt, mode))})
    assert response.status_code == 200
    result = response.json()
    assert result['label'] == 'shop'
    assert result['score'] == .8
    assert sum(result['probabilities'].values()) == pytest.approx(1)
    assert result['model'] == 'openai/clip-vit-base-patch32'
```

Тест проверяет код ответа, метку, score, сумму значений и имя модели. Он запускается для PNG, JPEG, WebP и PNG с прозрачностью. Здесь модель заменяется фиксированным предиктором, поэтому score 0,8 отличается от настоящего ответа выше. Другие тесты проверяют повреждённые файлы, лимиты и недоступность модели.

Успешный запуск 29 тестов в GitHub Actions

13 Постановка задачи рекомендаций

Рекомендательная система подбирает пользователю десять фильмов, которых нет в его истории оценок train. MovieLens 100K содержит 100 000 оценок 943 пользователей для 1682 фильмов. Релевантными считаются оценки 4–5. Демографические признаки не используются.

Подход	Реализация
Popular	Количество положительных оценок в train
Bayesian	Средняя оценка со сглаживанием 20 псевдооценками
Content	TF-IDF жанров и cosine с профилем понравившихся фильмов
Item-CF	Cosine по общим положительным оценкам; 40 соседей фильма
Hybrid	Среднее нормированных content и item-CF, вес 0,5

Два классических подхода и гибрид построены с NumPy, SciPy и scikit-learn. Popular и Bayesian служат простыми эвристическими baseline. Новому пользователю либо пользователю без положительной истории выдаётся popular. Все ранее оценённые фильмы, включая отрицательные оценки, исключаются.

Временное разделение и оценка

Для каждого пользователя последние 20% событий по времени оставлены в test; одинаковые временные отметки не разделяются. Получено 78 595 train-оценок и 21 405 test-оценок. Ranking-метрики усредняются по 915 пользователям с положительными test-оценками; остальные 28 в среднее не входят.

Кандидаты — весь каталог минус история train. Популярность, средние оценки и collaborative similarity используют только train. Жанры полного каталога считаются заранее доступными. 71 фильм в test отсутствует в train; на них приходится 17 положительных test-пар. Дополнительно рассчитан warm NDCG.

Разделение временное внутри каждого пользователя, но не является глобальным временным срезом: train-события одного пользователя могут быть позже test-событий другого. Для симуляции production нужен единый cutoff и учёт даты появления объекта. Параметры заданы до эксперимента и не оптимизировались по test.

Precision@10 — доля релевантных в списке; Recall@10 — доля найденных релевантных; NDCG@10 учитывает порядок с логарифмическим снижением веса; MAP@10 усредняет precision в позициях попаданий. Покрытие — доля каталога, встретившаяся в рекомендациях. Неизвестная оценка означает отсутствие наблюдаемой релевантности, а не доказанный dislike.

14 Результаты рекомендательных алгоритмов

Метод	P@10	R@10	NDCG@10	Покрытие
popular	0,0867	0,0752	0,1073	4.46%
bayesian	0,0568	0,0485	0,0601	3.57%
content	0,0328	0,0304	0,0403	28.18%
item_cf	0,1263	0,1310	0,1651	14.51%
hybrid	0,1142	0,1182	0,1497	25.68%

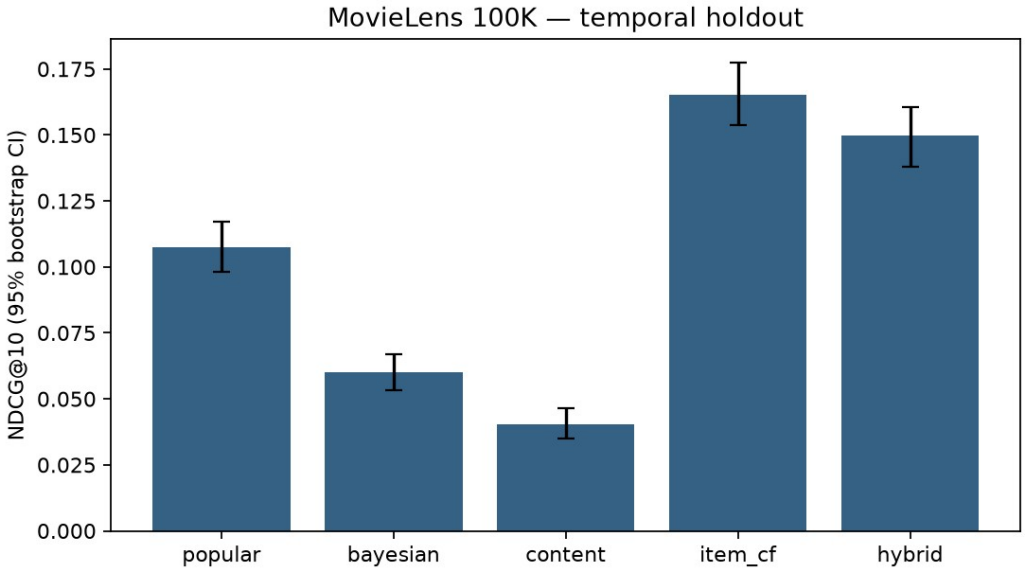


Рисунок 10 — NDCG@10 с 95% bootstrap-интервалами по пользователям

Item-CF показывает лучший NDCG@10: 0,1651 против 0,1073 у popular, относительное увеличение около 53,8%. Парная разность +0,0578 имеет 95% ДИ [0,0486; 0,0679]. Гибрид увеличивает покрытие каталога до 25,68% против 14,51% у item-CF, но уступает по NDCG. Content имеет широкое покрытие и низкую точность: одних жанров недостаточно для точного ранжирования.

Интервалы получены 1000 bootstrap-повторами по пользователям с seed=42 на фиксированном разделении. Они не учитывают перенос на иной каталог и аудиторию. Победитель выбран на том же test; для окончательного выбора алгоритма нужен отдельный контрольный период.

Ожидаемый эффект для сервиса

Гипотеза — персонализация повысит долю запусков фильмов и удержание. Offline-рост NDCG не доказывает рост CTR или выручки. Для A/B-теста предлагается рандомизация по пользователю: контроль popular против item-CF, основная метрика — доля запусков из блока рекомендаций, дополнительные — время просмотра, возврат через 7 дней и задержка p95. Размер выборки, полезный эффект и срок определяются до запуска.

15 Уязвимости и локальная языковая модель

Ограничения рекомендательной системы

Холодный старт требует fallback по популярности и стартового выбора интересов. Смещение в пользу популярных фильмов уменьшает разнообразие; следует измерять покрытие и качество по редким жанрам. Подставные оценки могут продвинуть объект, поэтому в реальном сервисе нужны лимиты, дедупликация и поиск аномалий. Эти защитные меры и атаки в эксперименте не реализованы.

MovieLens отражает оценки 1997–1998 годов и не представляет современный каталог. История оценок разрежена, а неизвестные объекты могут оказаться интересными. Для эксплуатации нужны свежие логи показов, мониторинг дрейфа и минимизация персональных данных. Плотная матрица сходств требует $O(I^2)$ памяти; при большом каталоге нужен разреженный kNN, ANN либо факторизация.

Развёртывание Qwen

Дополнительно в проекте запущена Qwen2.5-0.5B-Instruct с 0,49 млрд параметров и лицензией Apache-2.0. Небольшая модель подходит для ноутбука. Она работает локально через Transformers, с chat template, float32 и do_sample=False. Вход ограничен 2048 токенами, ответ — 512 новыми токенами.

Тестовый запрос: «Объясни в трёх предложениях, зачем нужны автоматические тесты API». Получен следующий неизменённый ответ:

1. Автоматические тесты API помогают проверить работу сервисов и систем, обеспечивая точность и стабильность.

2. Они позволяют быстро и эффективно обнаруживать ошибки или проблем с API-запросами.

3. Это также позволяет улучшать производительность и улучшать качество ответов от серверов.

Параметр запуска	Значение
Устройство	mps
Новых токенов	80
Время генерации	5.91 с
Скорость	13.54 токена/с

Qwen выдала три пункта, но написала «ошибки или проблем» и неточно связала тесты с повышением производительности. Тесты помогают найти ошибки, а ускорять сервис нужно отдельно. Этот пример показывает, что локальная генерация работает. Одного запроса недостаточно, чтобы оценить качество модели в целом; полученный код автоматически не исполняется.

16 Воспроизводимость и выводы

Порядок запуска

```
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements-dev.txt
python -m pytest -q
python -m image.evaluate
uvicorn api.main:app --host 127.0.0.1 --port 8000
python -m recsys.download_data
python -m recsys.evaluate
python -m recsys.recommender --user 1 --method item_cf
python -m llm.generate "Зачем нужны тесты API?"
```

Uvicorn нужно запустить в отдельном терминале. Для аудио и видео команды приведены в их README. При первом запуске скачиваются веса, затем они берутся из кеша. Скриншоты уже сохранены в репозитории. Для MovieLens есть скрипт загрузки, который записывает SHA256 архива.

Выводы

В результате получились работающие примеры для всех четырёх типов данных. В Images удалось собрать свой набор и получить macro-F1 0,8436 при заданном пороге 0,70. Классификатор доступен через FastAPI, а ответы сервера и основные расчёты проверяются тестами. На практике подготовка входных данных и проверка ошибок оказались такими же существенными частями проекта, как запуск готовой модели.

В RecSys item-CF превосходит popular по NDCG на выбранном временном разделении. Гибрид улучшает покрытие ценой части точности. Перенос результатов на реальный сервис требует нового временного периода и A/B-теста. Небольшие собственные наборы, визуальные смещения и ограничения оценочного протокола явно учтены при интерпретации результатов.

Источники и материалы

1. Задания: disk.yandex.ru/i/eMj2rXfaTH0c8Q; disk.yandex.ru/i/xdvx9uKuo9cpQA; disk.yandex.ru/d/k2wM2rv8ueTwFQ.

2. CLIP: https://huggingface.co/openai/clip-vit-base-patch32

3. RuBERT: https://huggingface.co/seara/rubert-tiny2-russian-sentiment

4. YAMNet: https://www.tensorflow.org/hub/tutorials/yamnet; ESC-50: https://github.com/karoldvl/ESC-50

5. YOLO11: https://docs.ultralytics.com/models/yolo11/; COCO: https://cocodataset.org/

6. FastAPI testing: https://fastapi.tiangolo.com/tutorial/testing/

7. GitHub Actions: https://docs.github.com/en/actions

8. MovieLens 100K: https://grouplens.org/datasets/movielens/100k/

9. Qwen: https://huggingface.co/Qwen/Qwen2.5-0.5B-Instruct

10. Исходные метрики и предсказания: каталоги results каждого модуля в репозитории.